Reviewer Pack — Minimal Rank of Noncrossing Partitions over Disjointness Com…

Entry f1b7db2e4a45 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 16:20:43 UTC

Minimal Rank of Noncrossing Partitions over Disjointness Communication Complexity

Entry ID: `f1b7db2e4a45` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-26 16:20:43 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Noncrossing Partitions) **Field B** (complexity object): Communication Complexity (Disjointness)

Statement:

> {'text': 'For any instance of the Disjointness problem with n variables, the minimal rank of the incidence matrix corresponding to a noncrossing partition of the set of subsets represented by the instance is Ω(n).', 'quantitative_relation': 'τ(DISJ_n) = Ω(n)', 'falsifier': 'For an instance of the Disjointness problem with n variables, there exists a noncrossing partition such that the minimal rank of its incidence matrix is O(1).'}

Rationale (proposer's reasoning):

> {'text': 'Noncrossing partitions are known to encode combinatorial information in a structured manner. If their incidence matrices have high rank, this might reflect complex interactions between subsets, which could be related to the communication complexity of Disjointness.', 'invariant_explanation': 'The structure of noncrossing partitions is well-understood and can be computationally realized, providing a potential bridge to the study of Disjointness communication complexity.'}

Taxonomy category: `COMM_DISJ` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `8fb02ddeb2a0a149`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For the Disjointness problem, if the minimal rank of the incidence matrix for any noncrossing partition is less than or equal to 3 times n/2 for at least 80% of the instances with n variables, then the conjecture is supported. If any seed produces a minimal rank greater than 10 or if support_fraction < 0.8, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "noncrossing partitions" AND "communication complexity" AND "disjointness" - "incidence matrices" AND "minimal rank" AND "algebraic combinatorics" - "τ(DISJ_n) = Ω(n)" AND "noncrossing partitions" AND "communication complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def incidence_matrix(instance):
        n = len(instance)
        matrix = [[0] * (1 << n) for _ in range(1 << n)]
        for i in range(1 << n):
            for j in range(1 << n):
                if all((i & (1 << k)) == (j & (1 << k)) for k in range(n)):
                    matrix[i][j] = 1
        return matrix

    def min_rank(matrix):
        rows, cols = len(matrix), len(matrix[0])
        rank = 0
        for i in range(rows):
            if any(matrix[j][i] != 0 for j in range(i, rows)):
                rank += 1
                for j in range(rows):
                    if matrix[j][i] != 0:
                        factor = Fraction(matrix[j][i], matrix[i][i])
                        for k in range(cols):
                            matrix[j][k] -= factor * matrix[i][k]
        return rank

    def generate_disjointness_instance(n):
```

```
        instance = []
        for _ in range(n):
            subset = random.sample(range(1, n + 1), random.randint(1, n))
            instance.append(subset)
        return instance

    n_values = [5, 10, 15, 20, 30, 40]
    total_rank = 0
    instances_tested = 0

    for n in n_values:
        for _ in range(5):  # Test each n with 5 different instances
            instance = generate_disjointness_instance(n)
            matrix = incidence_matrix(instance)
            rank = min_rank(matrix)
            total_rank += rank
            instances_tested += 1

    mean_value = total_rank / instances_tested
    support_fraction = (mean_value <= 3 * n_values[-1] / 2) and (mean_value > 0)

    return {
        "metric_name": "minimal_rank",
        "metric_value": mean_value,
        "instances_tested": instances_tested,
        "conjecture_holds": support_fraction,
        "counterexample": "" if support_fraction else f"rank={mean_value}, expected=3*{n_values[-1
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = all(result["conjecture_holds"] for result in results)

    if support_fraction:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction=1")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{results[seeds.index(first_failing_seed)]['cour
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means we cannot verify if the conjecture is supported or falsified according to the pre-registered criteria. | next: Re-run the test with increased time limits and ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16681
2	preregistration	ollama_remote	glm4:latest	0	0	6062
3	novelty	ollama_remote	glm4:latest	0	0	4801
4	novelty	ollama_remote	glm4:latest	0	0	5325
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18202
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10348
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9687
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8979
9	judge	ollama_remote	glm4:latest	0	0	14297

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 94380 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/f1b7db2e4a45.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f1b7db2e4a45.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/f1b7db2e4a45.tar.gz` (if generated)